

React: Data Fetching con Axios

Cheatsheet beginner per caricare dati da un'API in un componente React.

1. Il flusso generale

Quando un componente deve mostrare dati provenienti da un'API:

1. creiamo uno stato per salvare i dati;
2. eseguiamo la richiesta HTTP con Axios;
3. salviamo i dati ricevuti nello stato;
4. React aggiorna la pagina mostrando i nuovi dati.

```
Il componente viene montato
      ↓
useEffect esegue la richiesta
      ↓
Axios riceve la risposta
      ↓
setState salva response.data
      ↓
React esegue un nuovo render
```

2. API e richieste HTTP in breve

Un'**API** permette a due applicazioni di comunicare. Il frontend invia una richiesta a un **endpoint**, cioè un URL, e il server risponde normalmente con dati in formato JSON.

I metodi HTTP più comuni sono:

Metodo	Utilizzo
GET	Leggere dei dati
POST	Creare un nuovo dato
PUT / PATCH	Modificare un dato
DELETE	Eliminare un dato

In questa lezione ci concentriamo soprattutto sulle richieste **GET**.

Alcuni status code comuni:

Codice	Significato
200	Richiesta riuscita
404	Risorsa non trovata

Codice	Significato
500	Errore del server

3. Installare e importare Axios

Nel terminale, dalla cartella del progetto:

```
npm install axios
```

Nel file in cui vogliamo usarlo:

```
import axios from 'axios';
```

Una richiesta **GET** restituisce una Promise:

```
axios.get('https://jsonplaceholder.typicode.com/todos');
```

Axios inserisce la risposta in un oggetto. I dati che ci interessano si trovano quasi sempre in:

```
response.data;
```

4. Prima richiesta con `.then()` e `.catch()`

```
const loadTodos = () => {  
  axios  
    .get('https://jsonplaceholder.typicode.com/todos')  
    .then((response) => {  
      console.log(response.data);  
    })  
    .catch((error) => {  
      console.error(error);  
    });  
};
```

- `.then()` viene eseguito quando la Promise è completata con successo;
- il parametro `response` contiene la risposta ricevuta da Axios;
- `.catch()` viene eseguito quando la Promise fallisce;
- il parametro `error` contiene le informazioni sull'errore.

La richiesta è asincrona: JavaScript continua a eseguire il resto del codice mentre aspetta la risposta dell'API.

5. Caricare i dati all'avvio con `useEffect`

Una richiesta non deve essere eseguita direttamente nel corpo del componente: un aggiornamento dello stato causerebbe un nuovo render e quindi una nuova richiesta.

Usiamo invece `useEffect` con un dependency array vuoto:

```
import axios from 'axios';
import { useEffect, useState } from 'react';

export function TodoList() {
  const [todos, setTodos] = useState([]);

  useEffect(() => {
    axios.get('https://jsonplaceholder.typicode.com/todos').then((response)
=> {
      setTodos(response.data);
    });
  }, []);

  return <p>Todo caricati: {todos.length}</p>;
}
```

Perché `[]` è importante?

```
useEffect(() => {
  // Viene eseguito dopo ogni render
});
```

```
useEffect(() => {
  // Viene eseguito al montaggio del componente
}, []);
```

```
useEffect(() => {
  // Viene eseguito al montaggio e ogni volta che page cambia
}, [page]);
```

Senza dependency array può nascere un ciclo infinito:

```
richiesta → setState → render → richiesta → setState → render → ...
```

In sviluppo, se l'app usa `StrictMode`, React può eseguire un Effect due volte per aiutare a individuare problemi. Questo comportamento non avviene nello stesso modo nella build di produzione.

6. Usare la Promise dentro `useEffect`

`axios.get()` restituisce una Promise. Possiamo concatenare direttamente `.then()` e `.catch()` dentro l'Effect:

```
useEffect(() => {
  axios
    .get(API_URL)
    .then((response) => {
      setData(response.data);
    })
    .catch((error) => {
      console.error(error);
    });
}, []);
```

Ricordiamo il punto `.` prima di ogni metodo della catena:

```
axios.get(...) → .then(...) → .catch(...)
```

7. Dati, caricamento ed errore

Per dare un feedback chiaro all'utente possiamo usare tre stati:

```
const [todos, setTodos] = useState([]);
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState(null);
```

Esempio completo:

```
import axios from 'axios';
import { useEffect, useState } from 'react';

const API_URL = 'https://jsonplaceholder.typicode.com/todos?_limit=10';

export function TodoList() {
  const [todos, setTodos] = useState([]);
```

```
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  setIsLoading(true);
  setError(null);

  axios
    .get(API_URL)
    .then((response) => {
      setTodos(response.data);
    })
    .catch((error) => {
      setError(error);
    })
    .finally(() => {
      setIsLoading(false);
    });
}, []);

if (isLoading) {
  return <p>Caricamento...</p>;
}

if (error) {
  return <p>Errore: {error.message}</p>;
}

return (
  <ul>
    {todos.map((todo) => (
      <li key={todo.id}>{todo.title}</li>
    ))}
  </ul>
);
}
```

A cosa serve `.finally()`?

Il metodo `.finally()` viene eseguito sia in caso di successo sia in caso di errore. È quindi un buon posto per terminare il caricamento:

```
.finally(() => {
  setIsLoading(false);
});
```

Ordine consigliato del rendering condizionale

```
if (isLoading) {  
  return <p>Caricamento...</p>;  
}  
  
if (error) {  
  return <p>Si è verificato un errore.</p>;  
}  
  
return <DataList />;
```

In questo modo il JSX principale rimane più leggibile.

8. Mostrare una lista con `.map()`

I dati ricevuti non vengono mostrati automaticamente. Dobbiamo trasformarli in elementi JSX:

```
<ul>  
  {todos.map((todo) => (  
    <li key={todo.id}>  
      {todo.title} - {todo.completed ? 'Completato' : 'Da completare'}  
    </li>  
  ))}  
</ul>
```

Ogni elemento creato con `.map()` deve avere una `key` stabile e unica. Quando l'API fornisce un `id`, usiamo quello:

```
key={todo.id}
```

9. Parametri della richiesta

Axios permette di inviare query parameter usando `params`:

```
axios  
  .get('https://rickandmortyapi.com/api/character', {  
    params: {  
      page: 2,  
      name: 'rick',  
    },  
  })  
  .then((response) => {  
    console.log(response.data);  
  });
```

Axios costruirà un URL simile a:

```
https://rickandmortyapi.com/api/character?page=2&name=rick
```

Usare `params` rende il codice più leggibile rispetto alla costruzione manuale della stringa URL.

10. Paginazione

Per cambiare pagina:

1. salviamo la pagina corrente nello stato;
2. inseriamo `page` nelle dipendenze di `useEffect`;
3. inviamo `page` all'API;
4. modifichiamo lo stato con due pulsanti.

```
import axios from 'axios';
import { useEffect, useState } from 'react';

const API_URL = 'https://rickandmortyapi.com/api/character';

export function Characters() {
  const [characters, setCharacters] = useState([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setIsLoading(true);
    setError(null);

    axios
      .get(API_URL, {
        params: { page },
      })
      .then((response) => {
        setCharacters(response.data.results);
        setTotalPages(response.data.info.pages);
      })
      .catch((error) => {
        setError(error);
      })
      .finally(() => {
        setIsLoading(false);
      });
  }, [page]);

  if (isLoading) {
    return <p>Caricamento...</p>;
  }
}
```

```

    if (error) {
      return <p>Errore: {error.message}</p>;
    }

    return (
      <>
        <ul>
          {characters.map((character) => (
            <li key={character.id}>{character.name}</li>
          ))}
        </ul>

        <button disabled={page === 1} onClick={() => setPage(page - 1)}>
          Pagina precedente
        </button>

        <span>
          Pagina {page} di {totalPages}
        </span>

        <button disabled={page === totalPages} onClick={() => setPage(page +
1)}>
          Pagina successiva
        </button>
      </>
    );
  }

```

Quando `page` cambia, React esegue nuovamente l'Effect perché `page` si trova nel dependency array:

```

useEffect(() => {
  // Nuova richiesta con la pagina corrente
}, [page]);

```

11. Separare la chiamata API dal componente

Quando il componente cresce, possiamo spostare la richiesta in un file dedicato.

`api.js`:

```

import axios from 'axios';

const API_URL = 'https://rickandmortyapi.com/api/character';

export const fetchCharacters = (page) => {
  return axios.get(API_URL, {
    params: { page },
  });
}

```

```
});  
};
```

Nel componente:

```
import { fetchCharacters } from './api';  
  
useEffect(() => {  
  fetchCharacters(page)  
    .then((response) => {  
      setCharacters(response.data.results);  
    })  
    .catch((error) => {  
      setError(error);  
    });  
}, [page]);
```

Vantaggi:

- il componente si occupa principalmente dell'interfaccia;
- la chiamata API è più facile da leggere e riutilizzare;
- cambiare endpoint richiede meno modifiche al componente.

`export` e `import` funzionano perché con Vite usiamo i **moduli JavaScript**: ogni file può esportare valori e importare ciò che gli serve da altri file.

12. Errori comuni

Salvare l'intera risposta invece dei dati

```
// Errato nella maggior parte dei casi  
setTodos(response);  
  
// Corretto  
setTodos(response.data);
```

Dimenticare il valore iniziale corretto

Se poi usiamo `.map()`, lo stato deve iniziare come array:

```
const [todos, setTodos] = useState([]);
```

Dimenticare di gestire il risultato della Promise

```
useEffect(() => {  
  axios.get(API_URL).then((response) => {  
    setTodos(response.data);  
  });  
}, []);
```

Creare un re-render infinito

```
// Da evitare: viene eseguito a ogni render  
useEffect(() => {  
  axios.get(API_URL).then((response) => {  
    setTodos(response.data);  
  });  
});
```

Usare una variabile normale al posto dello stato

```
// React non aggiorna la pagina quando cambia  
let todos = [];  
  
// React esegue un nuovo render quando cambia  
const [todos, setTodos] = useState([]);
```

Dimenticare `.catch()` e il feedback per l'utente

Una richiesta può fallire per molti motivi: assenza di connessione, URL errato o problema del server. L'interfaccia deve poter mostrare almeno un messaggio di errore.

13. Checklist veloce

- ☐ Axios è installato e importato?
- ☐ I dati hanno uno stato con un valore iniziale corretto?
- ☐ La richiesta iniziale è dentro `useEffect`?
- ☐ Il dependency array contiene le dipendenze corrette?
- ☐ Ho usato `.then()` per gestire la risposta?
- ☐ Ho usato `.catch()` per gestire un eventuale errore?
- ☐ Sto salvando `response.data`?
- ☐ Ho gestito caricamento ed errore?
- ☐ Gli elementi creati con `.map()` hanno una `key`?
- ☐ I pulsanti della paginazione non superano i limiti?

Bonus: una piccola macchina a stati

Con tre stati separati (`data`, `isLoading` ed `error`) sono possibili combinazioni poco chiare, per esempio `isLoading === true` mentre esiste anche un errore.

Una **macchina a stati finiti** descrive invece una sola situazione alla volta:

```
idle → loading → success  
           oppure  
           error
```

Versione semplificata, senza `Object.freeze()` e senza `useCallback`:

```
import { useEffect, useState } from 'react';  
import { fetchActors } from './api';  
  
const initialState = {  
  step: 'idle',  
};  
  
export function Actors() {  
  const [state, setState] = useState(initialState);  
  
  useEffect(() => {  
    setState({ step: 'loading' });  
  
    fetchActors()  
      .then((data) => {  
        setState({  
          step: 'success',  
          data,  
        });  
      })  
      .catch((error) => {  
        setState({  
          step: 'error',  
          error,  
        });  
      });  
  }, []);  
  
  if (state.step === 'idle' || state.step === 'loading') {  
    return <p>Caricamento...</p>;  
  }  
  
  if (state.step === 'error') {  
    return <p>Errore: {state.error.message}</p>;  
  }  
  
  return (  
    <ul>  
      {state.data.map((actor) => (  

```

```
        <li key={actor.id}>{actor.name}</li>
      )}}
    </ul>
  );
}
```

Ogni variante contiene solo i dati che le servono:

```
{ step: 'idle' }
{ step: 'loading' }
{ step: 'success', data: [...] }
{ step: 'error', error: ... }
```

Questo pattern è utile quando la gestione dei vari stati diventa più complessa. Per un primo esercizio, gli stati separati **data**, **isLoading** ed **error** rimangono più semplici da capire e vanno benissimo.